

ARBORESCENCE DU PROJET

EDLoc — organisation du code : Backend (API Express) et Frontend (Next.js)

1. Organisation générale

Le code est organisé en deux dossiers distincts, Backend et Frontend, matérialisant la séparation entre l'API REST et l'application cliente retenue dans l'architecture (section 7 du cahier des charges). Chaque dossier possède ses propres dépendances (package.json) et sa propre configuration TypeScript ; les deux sont déployés sur Railway. Les schémas de validation Zod, écrits en TypeScript, sont présents des deux côtés : à l'entrée de l'API pour sécuriser les traitements, et côté client pour la validation des formulaires — c'est l'un des bénéfices concrets de la stack TypeScript unifiée.

2. Backend — API Express + Prisma

Le back-end suit une organisation en couches : les routes déclarent les points d'entrée, les contrôleurs orchestrent, les services portent la logique métier (création d'un état des lieux, copie entrée → sortie, génération du PDF, calcul de la comparaison), et les middlewares transversaux gèrent l'authentification JWT, le contrôle du rôle administrateur et la validation Zod. Le dossier prisma contient le schéma (traduction directe du MPD), les migrations versionnées et le script de seed qui crée le compte administrateur.

```
Backend/
├─ prisma/
│   ├─ schema.prisma          → modèles Prisma (traduction directe du MPD)
│   └─ migrations/            → migrations versionnées de la base
│       └─ seed.ts             → compte administrateur, données de démonstration
├─ src/
│   ├─ index.ts                → point d'entrée Express (middlewares globaux, routes)
│   └─ routes/                  → déclaration des routes par ressource
│       ├─ auth.routes.ts · biens.routes.ts · etats-des-lieux.routes.ts
│       ├─ pieces.routes.ts · elements.routes.ts · photos.routes.ts
│       └─ compte.routes.ts · admin.routes.ts
│   ├─ controllers/            → réception de la requête, appel du service, réponse
│   └─ services/                → logique métier
│       ├─ edl.service.ts (création, copie entrée→sortie) · signature.service.ts
│       └─ pdf.service.ts (génération) · comparaison.service.ts · stockage.service.ts
│   └─ middlewares/
│       ├─ auth.jwt.ts (vérification du jeton) · role.admin.ts (routes /admin)
│       └─ validate.ts (validation Zod des entrées) · erreurs.ts
│   └─ schemas/                 → schémas Zod (corps de requêtes, paramètres)
│       └─ utils/
├─ tests/
├─ .env.example                 → DATABASE_URL, JWT_SECRET, clés du stockage objet...
├─ package.json
└─ tsconfig.json
```

3. Frontend — application Next.js

Le front utilise l'App Router de Next.js avec trois groupes de routes qui reflètent exactement les trois zones de l'arborescence du site : (public), (bailleur) et (admin). Les groupes authentifiés sont protégés par un layout qui vérifie la présence et la validité du jeton ; le groupe (admin) exige en plus le rôle administrateur. Les composants shadcn/ui sont copiés dans le projet et adaptés aux tokens de la charte graphique via la configuration Tailwind (couleurs, rayons pilule, typographie Nunito Sans).

Frontend/

```

├─ app/                                → App Router Next.js
│   ├── (public)/
│   │   ├── page.tsx                  → page d'accueil (landing)
│   │   ├── connexion/page.tsx
│   │   ├── inscription/page.tsx
│   │   ├── mentions-legales/page.tsx
│   │   └── confidentialite/page.tsx
│   ├── (bailleur)/                  → groupe protégé (JWT requis)
│   │   ├── tableau-de-bord/page.tsx
│   │   ├── biens/page.tsx
│   │   ├── biens/nouveau/page.tsx
│   │   ├── biens/[id]/historique/page.tsx
│   │   ├── etats-des-lieux/nouveau/page.tsx
│   │   ├── etats-des-lieux/[id]/page.tsx      → détail + PDF
│   │   ├── etats-des-lieux/[id]/saisie/page.tsx
│   │   ├── etats-des-lieux/[id]/recapitulatif/page.tsx
│   │   ├── etats-des-lieux/[id]/signature/page.tsx
│   │   ├── etats-des-lieux/[id]/comparaison/page.tsx
│   │   └── compte/page.tsx
│   ├── (admin)/
│   │   └── admin/utilisateurs/page.tsx      → protégé par le rôle administrateur
│   └── layout.tsx                    → layout racine (Nunito Sans, fond crème)
├─ components/
│   ├── ui/                          → composants shadcn/ui adaptés à la charte
│   ├── edl/                        → saisie, pastilles d'état, signature, comparaison
│   ├── biens/                      → cartes, recherche, filtre commune
│   └── layout/                      → sidebar, navigation mobile, en-têtes
├─ lib/                             → client API (fetch), gestion du jeton, helpers
├─ schemas/                         → schémas Zod partagés avec l'API
├─ public/                          → logo, favicon
├─ tailwind.config.ts               → tokens de la charte (couleurs, rayons pilule)
├─ package.json
└─ tsconfig.json

```

4. Cohérences avec le reste du dossier

- Les groupes de routes du Frontend — (public), (bailleur), (admin) — reprennent les trois zones d'accès de l'arborescence du site.
- Les middlewares du Backend (auth.jwt, role.admin, validate) matérialisent les niveaux d'accès du tableau des routes de l'API.
- Le fichier prisma/schema.prisma est la traduction directe du MPD ; le seed crée le compte administrateur prévu au cahier des charges (section 4.6).